

Paso 1 — Validar rápidamente la nueva muestra

Antes de derivar nada:

```
SELECT
  COUNT(*) AS row_count,
  approx_count_distinct(party_id) AS party_count,
  approx_count_distinct(acct_id) AS account_count,
  MIN(data_date) AS minimum_data_date,
  MAX(data_date) AS maximum_data_date,
  COUNT(DISTINCT data_date) AS distinct_data_dates
FROM bu_frd.trades_behavior_sample_2024_2025;
```

Esperamos aproximadamente **24 data_date mensuales**, desde enero de 2024 hasta diciembre de 2025.

También confirma nuevamente la granularidad:

```
WITH duplicates AS (
  SELECT
    party_id,
    acct_id,
    data_date,
    COUNT(*) AS cnt
  FROM bu_frd.trades_behavior_sample_2024_2025
  GROUP BY
    party_id,
    acct_id,
    data_date
  HAVING COUNT(*) > 1
)
SELECT COUNT(*) AS duplicate_key_count
FROM duplicates;
```

Esperamos **0**.

Paso 2 — Crear las transiciones MOP 2024–2025

Ahora reproducimos la lógica que ya validamos, pero sobre los 24 meses.

```
DROP TABLE IF EXISTS bu_frd.trades_mop_transition_2024_2025;
```

```
CREATE TABLE bu_frd.trades_mop_transition_2024_2025
USING PARQUET
AS
```

WITH base AS (

SELECT

party_id,
acct_id,
data_date,
eff_dte,

acct_mop_cde,
acct_typ_cde,
porf_typ_cde,
kob_posn_1_cde,
kob_posn_2_cde,

open_dte,
clsd_dte,
paid_out_dte,

curr_bal_amt,
cr_lmt_amt,
high_cr_amt,
past_due_amt,

payt_amt,
payt_dte,

LAG(acct_mop_cde) OVER (
PARTITION BY party_id, acct_id
ORDER BY data_date
) AS previous_mop,

LAG(eff_dte) OVER (
PARTITION BY party_id, acct_id
ORDER BY data_date
) AS previous_eff_dte,

LAG(curr_bal_amt) OVER (
PARTITION BY party_id, acct_id
ORDER BY data_date
) AS previous_balance,

LAG(cr_lmt_amt) OVER (
PARTITION BY party_id, acct_id
ORDER BY data_date
) AS previous_credit_limit,

LAG(past_due_amt) OVER (

```

        PARTITION BY party_id, acct_id
        ORDER BY data_date
    ) AS previous_past_due

FROM bu_frd.trades_behavior_sample_2024_2025
),

mapped AS (

SELECT
    *,

    CASE
        WHEN previous_mop = '01' THEN 0
        WHEN previous_mop = '02' THEN 1
        WHEN previous_mop = '03' THEN 2
        WHEN previous_mop = '04' THEN 3
        WHEN previous_mop = '05' THEN 4
        WHEN previous_mop = '06' THEN 5
        ELSE NULL
    END AS previous_delinquency_level,

    CASE
        WHEN acct_mop_cde = '01' THEN 0
        WHEN acct_mop_cde = '02' THEN 1
        WHEN acct_mop_cde = '03' THEN 2
        WHEN acct_mop_cde = '04' THEN 3
        WHEN acct_mop_cde = '05' THEN 4
        WHEN acct_mop_cde = '06' THEN 5
        ELSE NULL
    END AS current_delinquency_level

FROM base
)

SELECT
    *,

    /* Cuenta activa en este snapshot */
    CASE
        WHEN (open_dte IS NULL OR open_dte <= data_date)
            AND (clsd_dte IS NULL OR clsd_dte > data_date)
        THEN 1
        ELSE 0
    END AS active_account_flag,

    /* Nueva actualización efectiva */
    CASE

```

```

    WHEN previous_eff_dte IS NOT NULL
      AND eff_dte <> previous_eff_dte
    THEN 1
    ELSE 0
  END AS effective_update_flag,

  /* Deterioro ordinal 01 -> 06 */
  CASE
    WHEN previous_delinquency_level IS NOT NULL
      AND current_delinquency_level IS NOT NULL
      AND current_delinquency_level >
        previous_delinquency_level
    THEN 1
    ELSE 0
  END AS dpd_deterioration_flag,

  /* Mejora ordinal */
  CASE
    WHEN previous_delinquency_level IS NOT NULL
      AND current_delinquency_level IS NOT NULL
      AND current_delinquency_level <
        previous_delinquency_level
    THEN 1
    ELSE 0
  END AS dpd_improvement_flag,

  /* Magnitud del cambio */
  CASE
    WHEN previous_delinquency_level IS NOT NULL
      AND current_delinquency_level IS NOT NULL
    THEN current_delinquency_level
      - previous_delinquency_level
    ELSE NULL
  END AS dpd_level_change,

  /* Entrada a adverse activo */
  CASE
    WHEN acct_mop_cde IN ('09','9B','8A','8B')
      AND COALESCE(previous_mop,"") NOT IN (
        '09','9B','8A','8B'
      )
    THEN 1
    ELSE 0
  END AS entered_active_adverse_flag,

  /* Adverse pagado/resuelto */
  CASE
    WHEN acct_mop_cde IN ('9P','8P')

```

```

        AND COALESCE(previous_mop,") <> acct_mop_cde
    THEN 1
    ELSE 0
END AS adverse_resolved_flag,

```

```

CASE
    WHEN acct_mop_cde = 'UR'
        AND COALESCE(previous_mop,") <> 'UR'
    THEN 1
    ELSE 0
END AS became_unrated_flag,

```

```

CASE
    WHEN previous_mop = 'UR'
        AND acct_mop_cde <> 'UR'
    THEN 1
    ELSE 0
END AS left_unrated_flag

```

FROM mapped;

Hay una ventaja importante: las transiciones de 2025 ahora tendrán acceso a diciembre/meses anteriores de 2024. Esto hace mucho más robusto el análisis longitudinal.

Paso 3 — Validar que 2025 siga comportándose igual

Antes de agregar a persona, quiero confirmar que al ampliar a 24 meses no rompimos nada:

```

SELECT
    SUM(
        CASE
            WHEN dpd_deterioration_flag = 1
                AND effective_update_flag = 1
            THEN 1 ELSE 0
        END
    ) AS deterioration_2025,

    SUM(
        CASE
            WHEN dpd_improvement_flag = 1
                AND effective_update_flag = 1
            THEN 1 ELSE 0
        END
    ) AS improvement_2025,

    SUM(

```

```

CASE
  WHEN entered_active_adverse_flag = 1
    AND effective_update_flag = 1
  THEN 1 ELSE 0
END
) AS active_adverse_2025

FROM bu_frd.trades_mop_transition_2024_2025

WHERE YEAR(data_date) = 2025;

```

No necesariamente será idéntico al análisis anterior, porque ahora enero de 2025 puede compararse correctamente con diciembre de 2024. De hecho, **esperaría que aparezcan algunos eventos adicionales en enero de 2025**.

Eso sería una mejora, no un problema.

Paso 4 — Crear la tabla **party × month**

Ahora sí:

```

DROP TABLE IF EXISTS bu_frd.party_trade_behavior_monthly_2024_2025;

CREATE TABLE bu_frd.party_trade_behavior_monthly_2024_2025
USING PARQUET
AS

SELECT
  party_id,
  data_date,

  /* Portfolio */
  COUNT(DISTINCT acct_id)
    AS total_accounts,

  SUM(active_account_flag)
    AS active_accounts,

  SUM(
    CASE
      WHEN clsd_dte IS NOT NULL
        AND clsd_dte <= data_date
      THEN 1 ELSE 0
    END
  )

```

```

) AS closed_accounts,

/* Exposure */
SUM(
  CASE
    WHEN active_account_flag = 1
    THEN COALESCE(curr_bal_amt,0)
    ELSE 0
  END
) AS total_active_balance,

SUM(
  CASE
    WHEN active_account_flag = 1
    THEN COALESCE(past_due_amt,0)
    ELSE 0
  END
) AS total_past_due_amount,

/* Delinquency state */
SUM(
  CASE
    WHEN active_account_flag = 1
    AND acct_mop_cde IN ('02','03','04','05','06')
    THEN 1 ELSE 0
  END
) AS delinquent_accounts,

SUM(
  CASE
    WHEN active_account_flag = 1
    AND acct_mop_cde IN ('03','04','05','06')
    THEN 1 ELSE 0
  END
) AS dpd_60plus_accounts,

SUM(
  CASE
    WHEN active_account_flag = 1
    AND acct_mop_cde IN ('04','05','06')
    THEN 1 ELSE 0
  END
) AS dpd_90plus_accounts,

/* Behavioral events */
SUM(
  CASE
    WHEN dpd_deterioration_flag = 1

```

```

        AND effective_update_flag = 1
    THEN 1 ELSE 0
END
) AS deteriorated_accounts,

SUM(
    CASE
        WHEN dpd_improvement_flag = 1
            AND effective_update_flag = 1
        THEN 1 ELSE 0
    END
) AS improved_accounts,

SUM(
    CASE
        WHEN entered_active_adverse_flag = 1
            AND effective_update_flag = 1
        THEN 1 ELSE 0
    END
) AS entered_active_adverse_accounts,

SUM(
    CASE
        WHEN adverse_resolved_flag = 1
            AND effective_update_flag = 1
        THEN 1 ELSE 0
    END
) AS adverse_resolved_accounts,

SUM(
    CASE
        WHEN became_unrated_flag = 1
            AND effective_update_flag = 1
        THEN 1 ELSE 0
    END
) AS became_unrated_accounts,

/* Severity */
MAX(
    CASE
        WHEN dpd_deterioration_flag = 1
            AND effective_update_flag = 1
        THEN dpd_level_change
    END
) AS max_dpd_jump,

SUM(
    CASE

```

```

        WHEN dpd_deterioration_flag = 1
        AND effective_update_flag = 1
        THEN dpd_level_change
        ELSE 0
    END
) AS total_dpd_deterioration_points,

/* Freshness */
COUNT(
    DISTINCT CASE
        WHEN effective_update_flag = 1
        THEN acct_id
    END
) AS effectively_updated_accounts,

/* Revolving / line-of-credit exposure only */
SUM(
    CASE
        WHEN active_account_flag = 1
        AND porf_typ_cde IN ('R','C')
        THEN COALESCE(curr_bal_amt,0)
        ELSE 0
    END
) AS revolving_balance,

SUM(
    CASE
        WHEN active_account_flag = 1
        AND porf_typ_cde IN ('R','C')
        THEN COALESCE(cr_lmt_amt,0)
        ELSE 0
    END
) AS revolving_credit_limit

```

FROM bu_frd.trades_mop_transition_2024_2025

GROUP BY
 party_id,
 data_date;

Aquí hice una corrección respecto a la versión anterior: **no mezclamos todas las cuentas para utilización**. Guardamos específicamente **revolving_balance** y **revolving_credit_limit**, porque **cr_lmt_amt** tiene mucho más sentido para **R/C** que para installment o mortgage.

Paso 5 — Agregar ratios

```
DROP TABLE IF EXISTS bu_frd.party_trade_behavior_monthly_2024_2025_v2;
```

```
CREATE TABLE bu_frd.party_trade_behavior_monthly_2024_2025_v2  
USING PARQUET  
AS
```

```
SELECT
```

```
*,
```

```
CASE
```

```
  WHEN active_accounts > 0
```

```
  THEN 1.0 * delinquent_accounts / active_accounts
```

```
END AS pct_accounts_delinquent,
```

```
CASE
```

```
  WHEN active_accounts > 0
```

```
  THEN 1.0 * dpd_60plus_accounts / active_accounts
```

```
END AS pct_accounts_60plus,
```

```
CASE
```

```
  WHEN active_accounts > 0
```

```
  THEN 1.0 * dpd_90plus_accounts / active_accounts
```

```
END AS pct_accounts_90plus,
```

```
CASE
```

```
  WHEN active_accounts > 0
```

```
  THEN 1.0 * deteriorated_accounts / active_accounts
```

```
END AS pct_accounts_deteriorated,
```

```
CASE
```

```
  WHEN active_accounts > 0
```

```
  THEN 1.0 * entered_active_adverse_accounts  
        / active_accounts
```

```
END AS pct_accounts_entered_adverse,
```

```
CASE
```

```
  WHEN active_accounts > 0
```

```
  THEN 1.0 * effectively_updated_accounts  
        / active_accounts
```

```
END AS pct_accounts_effectively_updated,
```

```
CASE
```

```
  WHEN revolving_credit_limit > 0
```

```
  THEN 1.0 * revolving_balance  
        / revolving_credit_limit
```

```
END AS revolving_utilization
```

```
FROM bu_frd.party_trade_behavior_monthly_2024_2025;
```

Paso 6 — Validaciones antes de construir ventanas

Primero:

```
WITH duplicates AS (  
  SELECT  
    party_id,  
    data_date,  
    COUNT(*) AS cnt  
  FROM bu_frd.party_trade_behavior_monthly_2024_2025_v2  
  GROUP BY party_id, data_date  
  HAVING COUNT(*) > 1  
)  
SELECT COUNT(*) AS duplicate_party_month_count  
FROM duplicates;
```

Debe dar 0.

Luego:

```
SELECT  
  COUNT(*) AS row_count,  
  COUNT(DISTINCT party_id) AS party_count,  
  MIN(data_date) AS minimum_data_date,  
  MAX(data_date) AS maximum_data_date,  
  COUNT(DISTINCT data_date) AS distinct_months  
FROM bu_frd.party_trade_behavior_monthly_2024_2025_v2;
```

Y finalmente quiero ver cobertura mensual:

```
SELECT  
  data_date,  
  COUNT(*) AS party_count,  
  ROUND(AVG(active_accounts),2) AS avg_active_accounts,  
  SUM(deteriorated_accounts) AS deteriorated_accounts,  
  SUM(entered_active_adverse_accounts) AS adverse_accounts  
FROM bu_frd.party_trade_behavior_monthly_2024_2025_v2  
GROUP BY data_date  
ORDER BY data_date;
```

Esta última validación es especialmente importante. Queremos comprobar que no haya un mes con una caída o incremento extraño debido al pipeline.

Después de estos resultados

El siguiente paso será probablemente el más importante hasta ahora: crear para cada `party_id + observation_date` variables como:

deteriorations_3m
deteriorations_6m
deteriorations_12m

adverse_events_3m
past_due_avg_3m
past_due_avg_12m

utilization_avg_3m
utilization_avg_12m

recent_3m_vs_prior_9m

Pero primero pásame los resultados del **Paso 3 y de las validaciones del Paso 6**. Si esos números se ven estables, construiremos las ventanas rolling sin volver a tocar la tabla gigantesca `trades`.